

**SIMATS ENGINEERING  
ASSESSMENT TOOL 2**

**COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY**

**COURSE CODE: CSA5112**

---

**COURSE OUTCOME COVERED**

**CO2:** Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

**Scenario-Based : 20%**

**Total Marks:**

**50**

**Annexure A**

---

***Code of Conduct:***

*I **T.Gokulnath(192521139)**(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***

**T.Gokulnath**

## Annexure A

### 1. Secure Password Storage in Web Applications

A web application stores user passwords and needs to protect them against database breaches.

A. Explain why storing plaintext passwords is insecure.

B. Suggest a secure method for storing passwords and justify your choice.

#### A. Why is storing plaintext passwords insecure?

Storing passwords in plaintext means the raw, human-readable password is saved directly in the database exactly as the user typed it. This is insecure for several reasons:

- Immediate total compromise on breach: if an attacker gains read access to the database (through SQL injection, an insider threat, a misconfigured backup, or a stolen disk), every user's password is exposed instantly with no additional effort required to decrypt or crack anything.
- Credential-stuffing risk across services: most users reuse the same password on multiple sites. A single plaintext leak therefore compromises the user's email, banking, and social accounts as well, not just the breached application.
- No defense-in-depth: plaintext storage provides zero resistance even to a low-skill attacker; there is no computational barrier, unlike a properly salted hash which forces the attacker to do expensive work per password.
- Insider misuse: employees or administrators with database access can view or misuse passwords directly, which also creates trust and compliance/regulatory issues (e.g., GDPR, PCI-DSS explicitly forbid plaintext password storage).
- Legal and reputational damage: regulators and users increasingly treat plaintext password storage as a sign of gross negligence, leading to fines and loss of trust after a breach disclosure.

#### B. Suggested secure method and justification

**Recommended method:** Store a salted, adaptive cryptographic hash of the password using a purpose-built password-hashing algorithm such as Argon2id (preferred), bcrypt, or scrypt — never a fast general-purpose hash like MD5 or SHA-256 alone.

How it works: when a user creates or changes a password, the server generates a long, random, unique salt for that user, concatenates it with the password, and passes it through the adaptive hash function with a configured work factor (cost parameter). Only the resulting hash and the salt are stored — never the password itself. During login, the same process is repeated on the entered password and the result is compared to the stored hash.

Justification:

- Irreversibility: a cryptographic hash is a one-way function, so even with full database access an attacker cannot mathematically recover the original password from the hash.

- Unique salting defeats rainbow tables: because each user has a different random salt, precomputed hash lookup tables become useless, and two users with the same password will have completely different stored hashes.
- Adaptive cost slows brute force: Argon2id/bcrypt are deliberately slow and memory-hard, so they can be tuned (as hardware improves) to keep cracking attempts computationally expensive — a fast hash like SHA-256 can be tested billions of times per second on a GPU, while Argon2id can be tuned to only a few hundred attempts per second.
- Industry and compliance alignment: OWASP, NIST SP 800-63B, and most security standards explicitly recommend Argon2id/bcrypt for password storage, making this the accepted best practice for CO2-level PKI/crypto system design reasoning.

## 2. Digital Signature for Document Authentication

**A company wants to ensure that official documents sent electronically are authentic and unaltered.**

**A. Explain how digital signatures can be used in this scenario.**

**B. Justify how this method ensures both integrity and non-repudiation.**

### A. How digital signatures can be used in this scenario

A digital signature scheme uses asymmetric (public-key) cryptography, typically RSA, combined with a cryptographic hash function, in the following sequence:

- Step 1 — Hashing: the sender computes a fixed-size message digest of the document using a secure hash function such as SHA-256.
- Step 2 — Signing: the sender encrypts (signs) this hash using their own RSA private key. This encrypted hash is the digital signature, which is attached to (or sent along with) the original document.
- Step 3 — Transmission: the document plus the digital signature are sent to the receiver, typically along with the sender's public-key certificate.
- Step 4 — Verification: the receiver independently computes the hash of the received document, then decrypts the attached signature using the sender's public key to recover the original hash. If the two hash values match, the document is verified as authentic and unaltered.

### B. How this ensures integrity and non-repudiation

**Integrity:** any modification to the document — even a single bit — changes the hash value computed at the receiver's end. Since this recomputed hash will no longer match the hash recovered from the decrypted signature, verification will fail, immediately revealing that the document was tampered with in transit.

**Non-repudiation:** the signature can only have been produced by whoever holds the sender's private key, which is kept secret and never shared. Since the receiver (or any third party) can

verify the signature using the sender's public key, the sender cannot later deny having signed the document — the mathematical link between the private key and the valid signature serves as legally binding proof of origin.

Together, these two properties make digital signatures suitable for official document authentication: the receiver gets cryptographic assurance both that the content is exactly what the sender produced (integrity) and that it genuinely originated from that sender, who cannot disown it later (non-repudiation).

### **3. Preventing Replay Attacks in Online Transactions**

**An online payment system is vulnerable to replay attacks where attackers resend previously captured transactions.**

**A. Explain how replay attacks occur.**

**B. Suggest a cryptographic technique to prevent such attacks and justify your answer.**

#### **A. How replay attacks occur**

In a replay attack, an attacker passively intercepts (eavesdrops on) a legitimate, correctly encrypted or authenticated transaction message travelling over the network — for example, a payment authorization request. Because the message itself is valid (it has a correct signature, MAC, or encryption), the attacker does not need to break the cryptography at all. Instead, the attacker simply captures a copy of this exact message and resends ("replays") it to the server at a later time, possibly multiple times. If the server has no way of distinguishing this repeated message from a fresh, legitimate one, it will process the transaction again — resulting in a duplicate payment, unauthorized fund transfer, or repeated login/session establishment — even though the attacker never learned the underlying secret key or password.

#### **B. Cryptographic technique to prevent replay attacks and justification**

**Recommended technique:** combine timestamps and/or nonces (number used once) with a Message Authentication Code (MAC) or digital signature, typically implemented as challenge-response authentication or a sequence-number/session-token scheme.

- Nonces: each transaction request includes a unique, randomly generated nonce. The server keeps a record of recently seen nonces (within a validity window) and rejects any transaction that reuses a nonce it has already processed.
- Timestamps: each request is signed together with the current timestamp; the server only accepts requests within a small allowed time window (e.g., 30–60 seconds), and rejects anything older or already processed within that window.
- Sequence numbers / one-time session tokens: each transaction increments a monotonic counter that both client and server track, or the server issues a single-use token per transaction that is invalidated immediately after use.

Justification: because the nonce, timestamp, or sequence number is bound cryptographically to the MAC/signature (it is included inside the data that gets hashed/signed), an attacker cannot

simply strip it out and reuse the message — doing so would invalidate the signature. Even though the attacker possesses a perfectly valid captured message, resending it will always fail either the freshness check (timestamp/nonce already used or expired) or the sequence check, so the replayed transaction is rejected without the server ever needing to break the underlying encryption.

#### **4. Secure Communication over Public Wi-Fi**

**A user accesses sensitive information using a public Wi-Fi network without encryption.**

**A. Identify the risks involved in this scenario.**

**B. Recommend a secure communication method and justify how it protects user data.**

##### **A. Risks involved in this scenario**

- Eavesdropping / packet sniffing: on an open, unencrypted Wi-Fi network, any nearby attacker can passively capture all traffic using tools like Wireshark, reading usernames, passwords, and sensitive data sent in the clear.
- Man-in-the-Middle (MITM) attacks: an attacker can position themselves between the user and the access point (e.g., via a rogue/evil-twin hotspot or ARP spoofing) to intercept, read, or modify traffic without either party noticing.
- Session hijacking: captured session cookies or tokens can let an attacker impersonate the user and take over an active logged-in session (e.g., email, banking, social media).
- DNS spoofing / phishing redirection: an attacker controlling the access point can redirect the user to fake login pages that harvest credentials.
- Malware injection: unencrypted downloads or HTTP pages can be modified in transit to inject malicious scripts or executables onto the user's device.

##### **B. Recommended secure communication method and justification**

**Recommended method:** use a Virtual Private Network (VPN) that establishes an encrypted tunnel (e.g., using IPSec or TLS-based protocols such as OpenVPN/WireGuard) for all traffic, combined with ensuring that individual sites are accessed only over HTTPS (TLS).

Justification:

- End-to-end encryption: a VPN encrypts all traffic between the user's device and the VPN server before it ever reaches the public Wi-Fi network, so anyone sniffing packets on that Wi-Fi sees only unreadable ciphertext, defeating eavesdropping.
- Protection against MITM: because the VPN/TLS tunnel is authenticated using certificates and key exchange (e.g., Diffie-Hellman within TLS), an attacker positioned on the local network cannot decrypt or tamper with traffic without detection, since doing so would break the certificate validation.

- HTTPS as a second layer: even without a VPN, HTTPS independently encrypts the application-layer session between the browser and the destination server using TLS, protecting login credentials and data specifically for that site.
- Defense-in-depth: using both a VPN and HTTPS together ensures that even if one layer were somehow compromised, sensitive data remains protected by the other, which is the recommended practice for accessing sensitive information over untrusted networks.

## 5. Data Integrity in Cloud Storage

**An organization stores critical data in a cloud server and wants to ensure that the data is not altered.**

**A. Explain how hashing can be used to verify data integrity.**

**B. Suggest an additional mechanism to enhance trust and justify your answer.**

### A. How hashing can be used to verify data integrity

Before uploading data to the cloud, the organization computes a cryptographic hash (a fixed-length digest) of the data using a secure hash function such as SHA-256. This hash value is stored securely by the organization, separately from the cloud provider (e.g., on a local system or a trusted third party), so it cannot be tampered with alongside the cloud copy.

Whenever the organization needs to verify that the stored data has not been altered — whether by an external attacker, a malicious insider at the cloud provider, or accidental corruption — it downloads the data (or asks the cloud provider to compute a fresh hash) and recomputes the hash locally using the same algorithm. If the newly computed hash matches the originally stored hash, the data is confirmed to be identical, bit-for-bit, to what was originally uploaded. Even a single-bit change in the stored data produces a completely different hash value (the avalanche effect), so any tampering — however small — is reliably detected.

### B. Additional mechanism to enhance trust and justification

**Recommended addition:** combine hashing with digital signatures (or a keyed MAC/HMAC) issued by the organization, and optionally use Merkle-tree-based proofs of data possession for large datasets.

- Digital signature over the hash: the organization signs the hash value with its own private key before storing it. This way, not only is the integrity of the data verifiable (via the hash), but the authenticity of the hash record itself is guaranteed — an attacker who tries to tamper with both the data and the stored hash cannot forge a valid signature without the private key.
- HMAC as a lightweight alternative: if a shared secret key exists between systems, an HMAC provides both integrity and authentication more efficiently than a plain hash, since it requires knowledge of the secret key to generate a valid tag.

- Merkle trees for large-scale data: for large datasets split into many blocks, organizing block hashes into a Merkle tree allows efficient, provable verification of individual blocks without re-hashing the entire dataset, and supports cloud-based "proof of data possession" schemes.

Justification: plain hashing alone only detects that data changed, but does not prove who is vouching for its correctness or prevent a sophisticated attacker (with database-level access) from replacing both the data and its stored hash together. Adding a digital signature or HMAC binds the integrity check to a secret or private key the attacker does not possess, so trust is anchored cryptographically rather than merely computationally, giving the organization strong, verifiable assurance against both accidental corruption and deliberate tampering by the cloud provider or third parties.

# **RUBRICS FOR CALCULATION:**

| <b>Criteria</b>              | <b>Weightage (%)</b> | <b>Level 4<br/>(Exemplary)</b>                                            | <b>Level 3<br/>(Proficient)</b>                    | <b>Level 2<br/>(Developing)</b>                   | <b>Level 1<br/>(Beginning)</b>                    |
|------------------------------|----------------------|---------------------------------------------------------------------------|----------------------------------------------------|---------------------------------------------------|---------------------------------------------------|
| Understanding of Scenario    | 20%                  | Demonstrates clear and complete understanding of the scenario and context | Shows good understanding with minor gaps           | Partial understanding ; misses key aspects        | Misinterprets or fails to understand the scenario |
| Identification of Key Issues | 20%                  | Accurately identifies all relevant issues and factors involved            | Identifies most key issues with minor omissions    | Identifies limited issues; some irrelevant points | Unable to identify key issues                     |
| Application of Concepts      | 25%                  | Applies appropriate concepts effectively to address the scenario          | Applies concepts with minor errors                 | Limited or partially correct application          | Unable to apply relevant concepts                 |
| Decision Making              | 20%                  | Makes well-reasoned, logical decisions with strong rationale              | Makes reasonable decisions with some justification | Makes weak decisions with limited justification   | No clear decisions made or rationale provided     |
| Clarity of Response          | 15%                  | Response is clear, structured, and logically presented                    | Generally clear with minor issues                  | Somewhat unclear or poorly structured             | Disorganized and difficult to understand          |